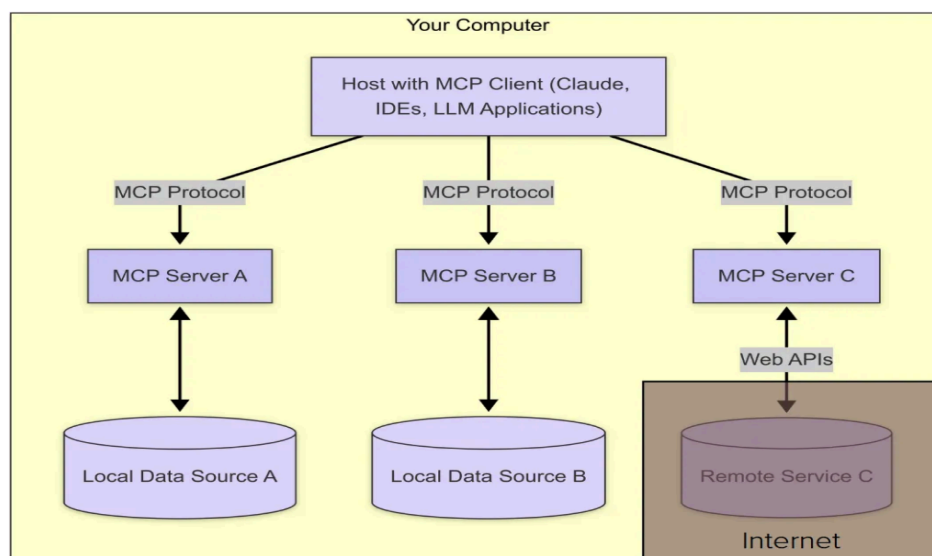
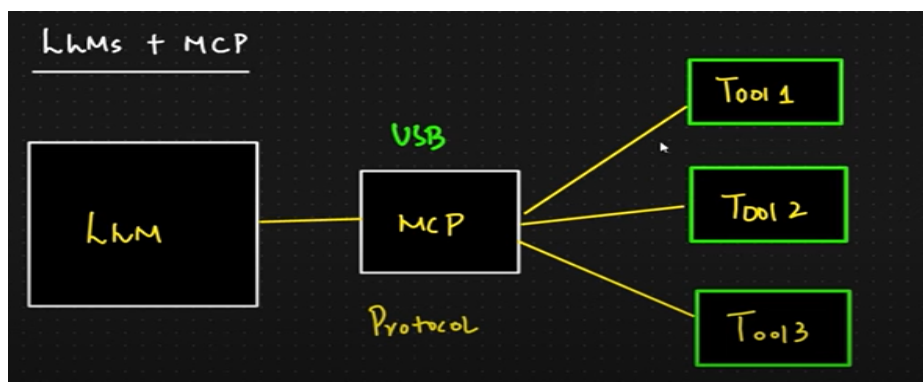


MCP (Model Context Protocol)

MCP is an open protocol that standardizes how applications provide context to LLMs. Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect your devices to various peripherals and accessories, MCP provides a standardized way to connect AI models to different data sources and tools.

Initially we worked with only LLM models, these have drawbacks like they cannot do dynamic tasks or cannot give info about current happenings. These issues were overcome by Agents and Tools, but suppose if we have requirements to handle hundreds of agents and Tools in our project then it will be difficult since it requires a lot of code to write and if any upgradation made wrt any tool then it demands upgradation in the code as well. To overcome these, MCP was introduced.

MCP acts as median between LLM and Tools and the tools have to adhere to the protocols set by MCP.



MCP has mainly 3 components:

- MCP Host
- MCP Client
- MCP Server

MCP Host- These can be programs like Claude Desktop (like chatgpt), VS code, IDEs, or AI tools that want to access data through MCP. MCP Hosts creates MCP Clients to interact with MCP Servers. The MCP Servers will be in turn connected to Code repositories or DB or API's etc. MCP Client will be communicating with the MCP server with the help of MCP Protocol.

MCP Host is like a LLM Application like Claude Desktop, Cursor, Windsurf like IDE or our own custom LLM Application. One host can connect to multiple servers using multiple MCP clients.

MCP Client: MCP client maintain the connection between MCP

server and the host application. This is a one-to-one connection. MCP host creates and manage MCP clients. Maintain security boundaries between servers. Clients maintain 1:1 connections with servers, inside the host application.

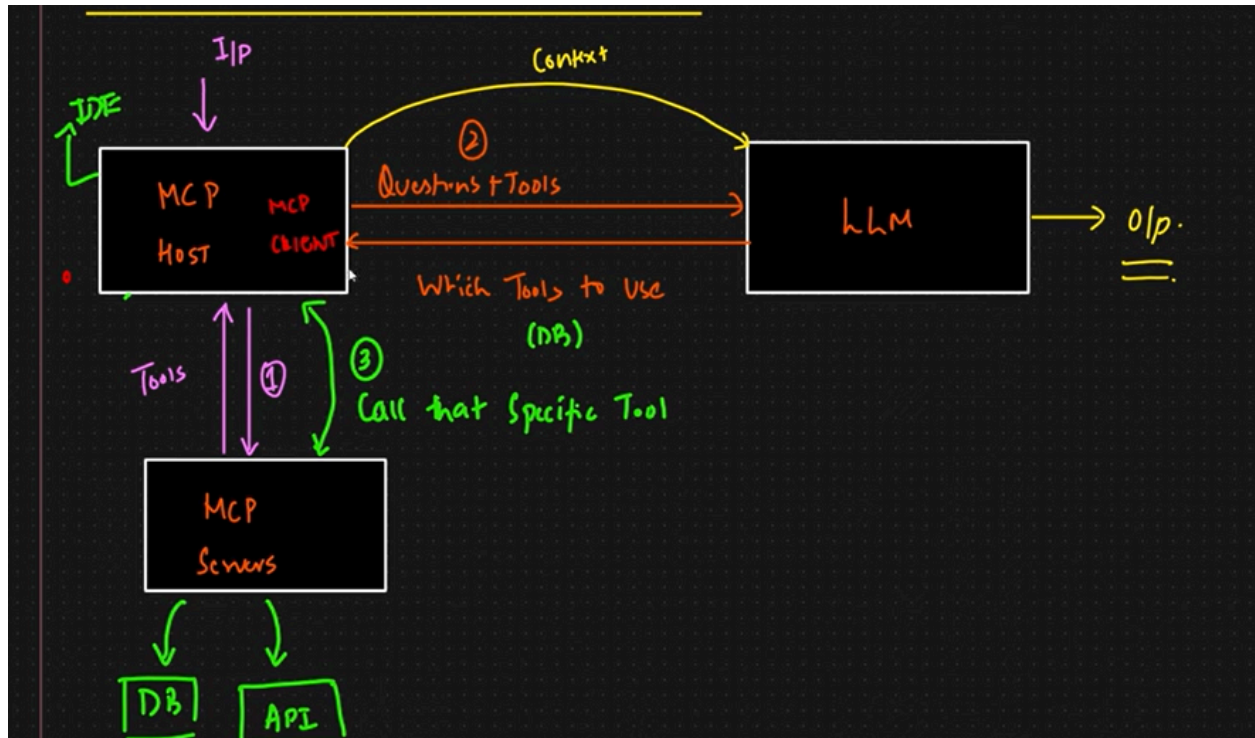
MCP Server: A program that follow MCP standards. MCP server contain set of **tools, resources and pre-written prompts** which can extent LLM abilities. External databases, API's etc can be connected to these servers. We can build our own servers, or we can use [existing MCP servers](#). Servers provide context, tools, and prompts to clients.

Can't We Do It Without MCP?

Absolutely we can connect tools and data sources to LLMs without MCP. Also, currently most of us do it without MCP. But the thing is when we do it without using MCP, we have

to write custom integration for every single tool and data source. We have to put additional effort into that. With MCP we can make it easy. We don't have to write custom integration for tools and data sources.

MCP uses client-server architecture. There are few main things we should know.



Steps:

- 1) When a user asks any questions as shown in the image above, the MCP client hits Server. The server will consist of multiple tools like DB, API's etc. Based on the request by MCP Client, the server will return all the tools to MCP Client.
- 2) The MCP client will now send the user Question along with the Tools which MCP Server has to LLM. The LLM, based on the Question, will tell MCP Client which all tools to use.
- 3) The MCP Client will now call all these Tools from MCP Server and send the context to LLM and then we get the required output.

=====

Hosts are LLM applications (like Claude Desktop or IDEs) that initiate connections.

Protocol layer

Handles message formatting. Defines how messages are structured and understood—like the grammar and etiquette of a conversation.

The protocol layer handles all message framing in both directions:

Client → Server (requests (to server), notifications)

Server → Client (responses (from client), notifications)

Transport layer

Handles the physical transmission of messages—like choosing between email, fax, or carrier pigeon.

Protocol without Transport:

Like having a letter with no address—it's well-written but won't reach anyone.

Transport without Protocol:

Like shipping a blank box—it arrives, but the recipient doesn't know what to do with it.

Together, they ensure messages are structured correctly and delivered reliably.

Example:

flowchart LR

Client-->|stdout|Server

Server-->|stdin|Client

Resources

Resources are a core primitive in the Model Context Protocol (MCP) that allow servers to expose data and content that can be read by clients and used as context for LLM interactions.

Resources represent any kind of data that an MCP server wants to make available to clients.

This can include:

- File contents
- Database records
- API responses
- Live system data
- Screenshots and images
- Log files
- And more

Each resource is identified by a unique URI and can contain either text or binary data.

Resource URIs

Resources are identified using URIs that follow this format:

[protocol]://[host]/[path]

For example:

file:///home/user/documents/report.pdf
postgres://database/customers/schema
screen://localhost/display1

The protocol and path structure is defined by the MCP server implementation. Servers can define their own custom URI schemes.

Prompts

Prompts enable servers to define reusable prompt templates and workflows that clients can easily surface to users and LLMs. They provide a powerful way to standardize and share common LLM interactions. Imagine a **restaurant (server)** creating a **menu (prompts)** so **customers (clients/LLMs)** don't have to explain how to cook every dish from scratch.

The server defines templates because:

It Knows the Tools

1. The server has access to specific tools/APIs (like a kitchen has recipes and ingredients).
2. Example: A server connected to a **weather API** might define a `get_weather` prompt template.

Example:

```
// Request
{
  method: "prompts/list"
}

// Response
{
  prompts: [
    {
      name: "analyze-code",
```

```
description: "Analyze code for potential improvements",
arguments: [
  {
    name: "language",
    description: "Programming language",
    required: true
  }
]
}
]
```

Tools

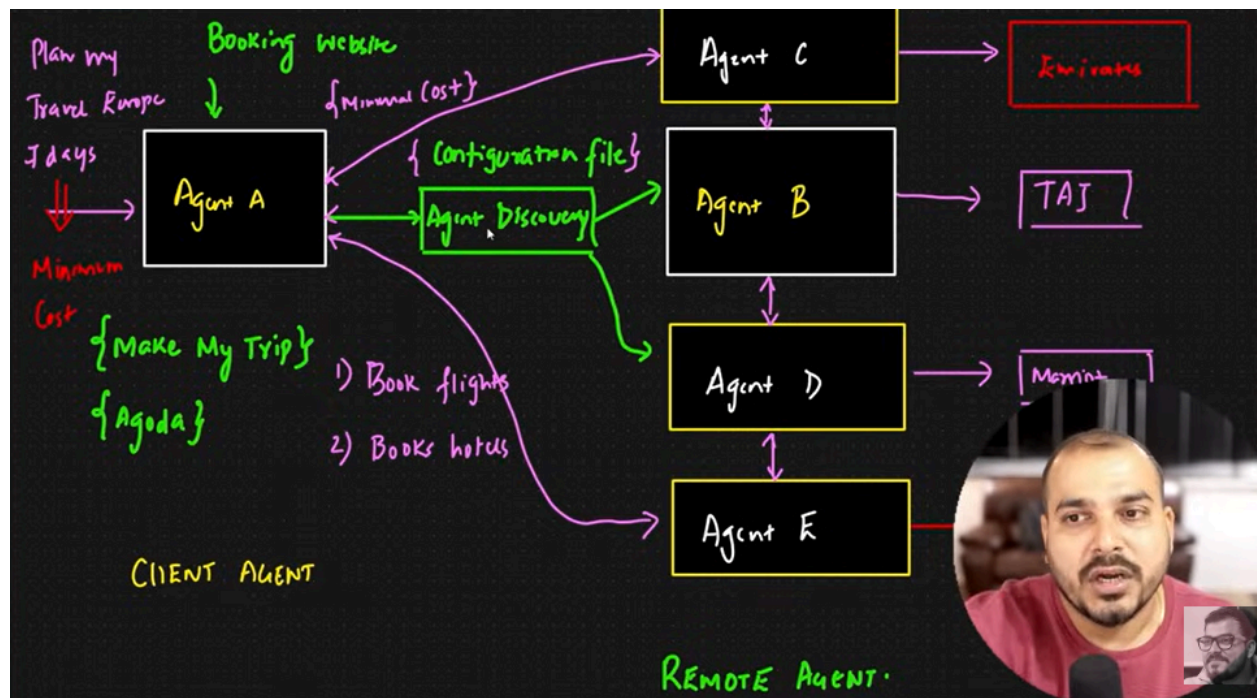
Tools are a powerful primitive in the Model Context Protocol (MCP) that enable servers to expose executable functionality to clients.

Tools are designed to be model-controlled, meaning that tools are exposed from servers to clients with the intention of the AI model being able to automatically invoke them (with a human in the loop to grant approval).

When to Use Each

- Need to perform an action? → Use a Tool
(e.g., translate text, fetch live weather).
- Need to access data? → Use a Resource
(e.g., load a PDF, read a settings file).

Agent2Agent Protocol



In the above architecture, 'Agent A' is called '**Client Agent**', it takes decisions based on the outputs of Agent B, Agent C, Agent D, Agent E which are called the '**Remote Agent**'. The Agent B,C,D,E also interact between each other.

Suppose our task is to prepare a Trip plan to Europe for 7 days at minimum cost. To complete this task different agents belonging to different companies are used. Agent C belongs to Emirates, Agent E belongs to Air India. These two agents send their output to Agent A and Agent A decides which flight has minimum cost. Agent B belongs to Taj, Agent D belongs to Operia, these two agents send their output to Agent A and Agent A decides which hotel has the minimum cost.

A2A design principles:

1) Embrace Agentic Capabilities: A2A focuses on enabling agents to collaborate naturally between them as shown in agents in remote agents in above eg.

2) Build on existing standards: The protocol is built on top of existing, popular standards including HTTP, SSE, JSON-RPC, which means it's easier to integrate with existing IT stacks businesses already use daily.

3) Secure by default: A2A is designed to support enterprise-grade authentication and authorization, with parity to OpenAPI's authentication schemes at launch.

4) Support for Long-Running Tasks: Handle tasks that span seconds to days, with real-time updates. Also Human-in-the-loop support (e.g., approvals, interruptions).

5) Modality Agnostic: Enable agents to work beyond text (audio, video, UI). Content-type flexibility (JSON, images, binary data).

How A2A works:

A2A facilitates communication between a "client" agent and a "remote" agent. A client agent is responsible for formulating and communicating tasks, while the remote agent is responsible for acting on those tasks in an attempt to provide the correct information or take the correct action. This interaction involves several key capabilities:

Capability discovery: Agents can advertise their capabilities using an "Agent Card" in JSON format, allowing the client agent to identify the best agent that can perform a task and leverage A2A to communicate with the remote agent.

Collaboration: Agents can send each other messages to communicate context, replies, artifacts, or user instructions.

MCP V/S A2A

1) MCP (Model Context Protocol)

Purpose: Connects a single AI agent to external tools, data, and services.

Key Function: Provides a standardized way for AI models to access context, including data, tools, and dynamic information.

Example1: Travel Planning with MCP

1. User asks AI: "Book a flight to Paris next week."
2. AI uses MCP to:
 - Access your calendar (via Google Calendar API) to check availability.
 - Fetch flight options (via Kayak/Expedia API).
 - Retrieve your passport details (from a secure company database).
3. Result: The AI acts alone—it doesn't need other agents, just access to tools/data.

Example2: Allows an AI assistant to access a knowledge base, search the web, or interact with a calendar app.

Complementary to A2A:

MCP focuses on connecting individual models to resources, while A2A focuses on how agents interact with each other.

2) A2A (Agent-to-Agent Protocol)

Purpose: Enables multiple AI agents to collaborate, delegate tasks, and share information between each other.

Key Function: Provides a standardized way for AI agents to discover each other, communicate, and delegate tasks, even if they are built by different vendors or on different platforms.

Example: Corporate Event Planning with A2A

1. User asks AI: "Organize a team offsite in Paris."
2. Agents collaborate via A2A:
 - Travel Agent: Finds flights/hotels (uses MCP to access booking APIs).
 - Calendar Agent: Checks attendee availability (syncs with Outlook/GCal).
 - Budget Agent: Approves costs (connects to accounting software).
 - Logistics Agent: Schedules ground transport (messages the Travel Agent for pickup times).
3. Result: Agents negotiate autonomously, updating each other in real time.

Complementary to MCP:

A2A enables agent collaboration, while MCP connects individual models to external resources.

In essence:

- **MCP connects AI to tools and data.**
- **A2A connects AI to other AI.**

